ការសរសេរអក្សរខ្មែរស្វ័យប្រវត្តិ

Khmer OCR — Built From Scratch

A text corpus, a synthetic structured-document dataset, and a trained layout-aware OCR model — designed together as one pipeline, for a script with no inter-word spacing and complex combining-mark shaping.

01 LM Text Corpus

02 Grounded OCR Dataset

03 Two-Stage OCR Model

01 Why This Is Hard

Khmer script, technically

- No spaces between words — word-level boxes are simply undefined
- Coeng (subscript) consonants and stacked vowel combining marks must be rendered and normalized in an exact, idempotent order
- Plain bitmap renderers (e.g. Pillow) do not shape Khmer correctly at all

Data reality

- Low-resource language: little clean, licensed, structured text at scale
- No existing labeled dataset for *structured-layout* Khmer OCR (headings, tables, forms, charts, signatures) — only flat line text

The approach

Build all three layers in one repo, each verified before the next depends on it:

- **Corpus** — source, clean, dedupe and license-track Khmer text
- **Render** — turn that text into labeled document images via a real browser engine, so shaping is correct by construction
- **Train** — a from-scratch model that reads structure and text together, in a format an OCR foundation model already understands

02 Three Deliverables, One Repository

01 Khmer LM Text Corpus `data/corpus/lm_corpus/`

Broadly sourced, normalized, deduplicated, quality-filtered, license-tracked Khmer text. Reusable on its own to train a future Khmer LLM — and it's the text source for deliverable #2.

02 Structured-Layout OCR Dataset `data/synthetic/` + `data/manifests/`

Synthetic document images with grounded-markdown labels in DeepSeek-OCR-2's `<|grounding|>` format — text *and* layout (headings, tables, forms, charts, signatures) with pixel-aligned boxes, and zero manual annotation.

03 From-Scratch OCR Model `src/train/` , `src/detect/` , `src/recognize/`

A two-stage detector + recognizer pipeline (plus an original single-model encoder-decoder baseline), trained from random init on the synthetic dataset above, targeting real scanned/photographed documents.

03 End-to-End Pipeline

STAGE 1 — TEXT CORPUS

sources.yaml → registry / fetch → clean_normalize → dedup → quality_filter → **lm_corpus/**

STAGE 2 — SYNTHETIC OCR DATASET

lm_corpus + fonts + layouts.yaml → render_playwright → augment → to_grounded_markdown → **synthetic/ + manifests/**

STAGE 3 — MODEL TRAINING

manifests + tokenizer → detector (FCOS) → line split → recognizer (SVTR) → **assemble → markdown**

04 Deliverable 1 — The LM Text Corpus

Sourcing constraint

`datasets>=4` dropped support for loading-script HF datasets — script-based sources (CC-100, mC4, MADLAD-400, OSCAR-2301) simply cannot load anymore.

Only **Parquet** sources work: FineWeb-2 (`khm_Khmr`), Glot500 (`khm_Khmr`), Wikimedia Wikipedia. Every source's status and reason for exclusion is recorded in `sources.yaml` .

Provenance, end to end

A single `Doc` dataclass and JSONL shard format carries source, license, and lineage through every stage. Intermediate dirs are fully regenerable — only `raw/` and `lm_corpus/` are worth keeping.

Pipeline stages

- **registry --fetch** — pull from configured sources (HF datasets, dumps, scrape), license-gated
- **clean_normalize** — text cleanup + Khmer-specific normalization
- **dedup** — exact + MinHash near-duplicate removal
- **quality_filter** — LM-grade heuristic filtering, with a rejected-sample audit trail
- **package_lm** — final `lm_corpus/` + auto-generated dataset card

05 Deliverable 2 — Rendering, Not Compositing

HTML/CSS → Playwright (Chromium) → DOM boxes

Pages are built as real HTML/CSS and rendered through Chromium specifically because it shapes Khmer correctly via HarfBuzz — coeng subscripts and stacked vowels land in the right visual position. Bitmap renderers do not do this.

Perfect alignment, zero manual labeling

Bounding boxes come straight from the DOM — `getBoundingClientRect` plus `Range` line rects — so image and label are pixel-aligned by construction, and **reading order = DOM order**. No human ever draws a box.

Fonts & diversity

- Each chosen font is base64-embedded as an `@font-face` data URL, so it applies regardless of page origin
- `fonts.yaml` deliberately up-weights hard decorative fonts (Bayon, Dangrek, iSeth First…) — exactly where OCR tends to fail
- Layouts, augmentation and page style are entirely config-driven (`config/*.yaml`) — no behavior is hard-coded

06 Label Format — Grounded Markdown

One grounded reference per leaf, in DOM order

```
<|ref|>{markdown text}<|/ref|><|det|>[[x1,y1,x2,y2]]<|/det|>
```

Coordinates are normalized to `[0, 999]`. This is the exact training-target format and matches DeepSeek-OCR-2's own grounding prompt: `<image>\n<|grounding|>Convert the document to markdown.`

Structure, reconstructed

- Tables rebuilt from `data-tbl/row/col` attributes
- Headings, lists, captions all preserved as markdown

Box granularity

- Line / block-level, deliberately — word boxes are ill-defined in spaceless Khmer

07 Two Non-Negotiable QA Gates

Gate 1

roundtrip_check

The text fed into a page must equal the text decoded back from its label, after normalization — **must be 100%** before any large generation run.

Catches Unicode/coeng-ordering bugs and silent text loss. Backed by `khmer_utils.normalize()`, the correctness-critical function that canonicalizes combining-mark order and strips zero-width controls — it must stay strictly **idempotent**, and is applied identically to rendered text and labels.

Gate 2

visual_qa / box_sanity

A contact sheet over every font × template, eyeballed by a human for shaping bugs and `.notdef` boxes — failure modes loss curves never reveal. `box_sanity` checks boxes are in-bounds and correctly ordered.

Why this matters

> "roundtrip_check catches Unicode-ordering bugs and text loss; visual_qa catches shaping bugs and `.notdef` boxes that loss curves never reveal. Run both before any large generation."

There is no test framework for this project — correctness is verified entirely by these gates running over the generated labels, plus small `--smoke` runs.

08 Deliverable 3 — Model Architecture

Swin + BART encoder-decoder, random init

Built via HF `VisionEncoderDecoderModel` for layer plumbing only — no pretrained weights are loaded anywhere. Everything is trained from scratch on the synthetic dataset.

SentencePiece tokenizer

- Unigram, vocab 22k, trained on the LM corpus + ~50MB English Wikipedia
- **1000 coordinate tokens** `<0>…<999>` registered as user-defined symbols, so each normalized bbox coordinate tokenizes to a single id — keeps grounded sequences ~4× shorter

Two size profiles, one `train.py`

PROFILE	PARAMS	INPUT	HARDWARE
`--single`	~50M	768px	1× RTX 4070 Ti (12GB)
`--parallel`	~250M	1280px	3× A5000 24GB, DDP

`accelerate` handles single-GPU vs. DDP transparently — one training script, config-selected geometry.

09 From One Model to a Two-Stage Pipeline

Why split detection from recognition

The original single encoder-decoder baseline reads an entire page end-to-end. The current architecture splits the job in two — a **detector** that finds structured regions, and a **recognizer** that reads text inside each one — then **assembles** the results back into structured markdown.

Detector: DBNet → FCOS

V1–V3 used a DBNet segmentation detector. V4 replaced it with a **from-scratch FCOS object detector** — region-level boxes (overlap allowed), so it can represent a `table` region enclosing its cells, `chart` vs. `image`, `signature`, etc.

Key fix: **ATSS assignment** — vanilla FCOS starves extreme-aspect-ratio document regions (e.g. a 900×40 title gets no grid cell). Caught by an overfit smoke gate before any large training run.

Structure, restored

A later audit found the detector was perfect on boxes but had dropped `block_type` entirely — flat output, no headings or tables. Fix: a per-pixel/per-region **class head**, 16 region classes, feeding class-aware line→block merging and real markdown emission (headings, lists, tables).

10 Detector Class Taxonomy (16 Classes)

Text-bearing (recognized)

title · heading · subheading · text · list_item · caption · table · table_head · table_cell · form_label · form_value

Non-text (cropped, not recognized)

image · chart · signature · hand_drawing · formula

The hard part was the rare classes

Early runs measured a ~260:1 class imbalance — chart, formula, hand_drawing were under 1% of instances and the detector simply couldn't learn them (hand_drawing F1 started at **0.00**). Fixing this required deliberate data engineering, not just more training — see the V5 data engine, next.

11 Closing the Synthetic → Real Gap

DPI mismatch was the dominant failure — not scan appearance

Experiment: upscaling a real scan to synthetic training DPI made the *unmodified*, no-augmentation recognizer read it correctly. The recognizer was trained on crops **downscaled** from high-DPI renders, but small real scans were being **upscaled** to the same target size (blurry, out-of-distribution). Fix: `normalize_dpi()` upscales narrow inputs before detection so real crops get downscaled like training data did.

Line-merging on tight real spacing

Detector was perfect on synthetic (det/GT line ratio 1.00) but merged lines on tight-spaced real documents. Fix: **vertical-squash augmentation** (0.55–0.95×) simulates tight spacing from existing generously-spaced renders — no re-render needed.

Headings collapsing to plain text

The class head overfit synthetic heading *appearance* and misread real headings as body text. Fix: a **geometry-aware structure refiner** reconciles the learned class with domain-invariant geometry (centering, relative height, width fraction) at assembly time.

Also: scan-realistic recognizer augmentation

Tint / noise / blur / photocopier-style artifacts added to recognizer training crops to close the remaining appearance gap between rendered pages and phone/scanner captures.

12 V5 — The Generalization Gap, Measured

The synthetic val set was saturated

V4 baseline: **meanF1@0.5 = 0.981** on its own synthetic distribution — the model had mastered it, and synthetic val could no longer drive progress. A new eval harness (`det_eval.py` , `real_battery.py` , 5 fixed hard real pages) was built first, specifically to stop trusting that number.

What V5 added

- **Compositional layout engine** — ~50% of pages built from packed element bands, not fixed templates
- **Style diversity** — colored text/fonts, ruled paper, pen-ink forms, phone-light gradients, photocopier banding
- **Class-balance gating** — `stats --gate` hard-fails a render if any class share drops under its floor

The gap, quantified (400-page eval)

MODEL ON DISTRIBUTION	F1@0.5	F1@0.75
V4 model → V4 (own) val	**0.981**	0.951
V4 model → V5 (harder) val	**0.744**	0.693
V5 model → V5 val	**0.957**	0.912

V4's 0.981 was narrow-distribution overfitting. Rare classes on the harder V5 distribution: chart **0.973**, formula **0.981**, hand_drawing **0.965** — versus ≈0–0.83 in the V4 era.

13 Serving Target: CPU-Only Inference

Constraint

Production inference runs on a modest 4-core cloud CPU, targeting **1–3 s/page**. The 3× A5000 box is a **training-only** accelerator — there is no GPU in the serving path.

Measured baseline (4 threads)

PAGE TYPE	LINES	TIME @1024
Real legal (sparse)	19–22	2.16 s
Scientific paper	34–36	3.38 s
Book page (dense)	40–61	**4.83 s**

Recognizer cost is linear per line (60–80ms) and dominates dense pages; its cost lives in the shared encoder, not the decoder.

What was tried

- Detector downscale 1024→768: **2.2× faster** but **drops lines** on dense pages — not free
- CTC single-shot decode vs. attention-AR: barely faster, **loses 9pts exact-match**
- ONNX Runtime fp32: modest **~1.2×** (PyTorch's CPU conv already well-tuned)
- ONNX **dynamic INT8**: **13–14× SLOWER** — wrong tool for conv + small matmuls

The real levers (A5000's job)

Retrain the detector natively at 768px (recover the accuracy lost by post-hoc downscaling), shrink the recognizer's shared encoder, and use **static/calibrated** INT8 — not dynamic — where the target CPU supports it.

14 Current Status

Timeline

- **V1–V3** — end-to-end baseline, then DBNet detector + recognizer, then real-image 15-class dataset. Frozen; all later work is additive.
- **V4** — from-scratch FCOS object detector (region-level, 16 classes) replaces DBNet; ATSS assignment fix; structure/markdown restored.
- **V5 data engine** — eval harness, compositional layouts, style diversity, class-balance gating, box-accuracy fixes (Phases A–F).
- **V5 detector r3** — GO for A5000: 350K compositional render → `--parallel` train in progress.
- **Geometry-aware structure refiner** — just landed; needs visual validation on real docs once GPU is free.

Frozen vs. active

- ✓ V1/V2/V3 checkpoints — frozen, kept as baselines
- ✓ Corpus pipeline & QA gates — stable, in continuous use
- ✓ V5 data engine (code) — complete, all gates passing
- ○ V5 350K parallel render + train — in flight on A5000
- ○ Recognizer refresh on scan-realistic data
- ○ CPU-serving path (static INT8 + lean recognizer)

SUMMARY

One Repo: Corpus → Dataset → Model

A Khmer LM text corpus with tracked provenance and licensing; a structured-layout OCR dataset rendered through a real browser engine for correct shaping and pixel-perfect, zero-annotation labels; and a two-stage detector + recognizer OCR model — each stage gated by verifiable QA, each generation informed by a measured (not assumed) real-document gap.

Config-driven · every behavior in YAML

Provenance-tracked · every source licensed

Gate-verified · no untested generation